1. 일단 BConv가 뭘 하는 하드웨어인지부터

네가 전에 이해했던 BConv 수학을 RTL에 대응시키면 훨씬 쉽다.

예를 들어 현재 ciphertext residue가

₩[([x]_{q_0},[x]_{q_1},₩dots,[x]_{q_L}) ₩]

로 있고, 이것을 새로운 modulus ₩(p_j₩)에 대한 residue

₩[[x]_{p_j} ₩]

로 만들어야 한다고 하자.

BConv의 전형적인 형태는 대략

₩[[x]_{p_j} = ₩sum_{i=0}^{L} [x]_{q_i} ₩cdot C_{i,j} ₩pmod {p_j} ₩]

처럼 볼 수 있어.

여기서 ₩(C_{i,j}₩)에는 CRT reconstruction에 필요한 미리 계산된 상수들이 들어간다.

즉 하드웨어 입장에서 보면 결국:

q0 residue —× constant—┐

q1 residue —× constant—┤

q2 residue —× constant—┤

... ├— modular accumulation → p0 residue

qL residue —× constant—┘

이런 **multiply-accumulate** 연산이 핵심이야.

그리고 네 CU.v가 정확히 이걸 수행하려고 만들어진 구조다.

---

2. 전체 RTL 계층은 이렇게 보면 된다

현재 네가 올린 파일 기준으로 **BConv 핵심 hierarchy**를 그리면:

Bconv_engine.v

└—— Bconv ← BConv top / controller

  |

```
├─── FSM
|       ├─── IDLE
|       ├─── STEP1
|       └─── STEP2
|
└─── CU.v × 16                  ← NUM_CU = 16
        |
        ├─── buffer[0:16]
        |
        ├─── shoup_mult.v       ← modular multiplication
        |
        └─── modular accumulator
                ├─── temp_sum
                ├─── modadd
                └─── cu_accum
```

이게 **일단 BConv 공부할 때 봐야 할 전부**라고 생각해도 된다.

반면 폴더에 같이 있는:

modular_unit.v

```
├─── modular_add
├─── modular_sub
├─── barrett_mult
└─── automorphism 관련 연산
```

modadd.v

modsub.v

barrett_mult.v

modmult_mgmr.v

automorphism.v

sram_1024w_32b.v

등은 당장 BConv부터 이해하려고 할 때 같이 파고들 필요 없다.

NTTU 때 barrett_mult 내부부터 들어가지 말라고 했던 것과 똑같아. 그때도 처음 목표는 butterfly가 수식을 정해진 latency로 구현한다는 데까지만 보라고 했었다. rtl

---

3. 그런데 왜 CU가 16개인가?

Top에:

parameter NUM_CU = 16

이 있고,

generate

```
for (i = 0; i < NUM_CU; i = i + 1) begin : CU_inst_loop


    CU CU_inst (
        ...
        .coeff(coeff[i]),
        .param(param[i]),
        .q_in(q_in[i]),
        .ratio_precomp(ratio_precomp[i]),
        ...
    );


end
```

endgenerate

가 있다.

즉 실제로

CU0

CU1

CU2

...

CU15

16개를 병렬 instantiate한다.

여기서 중요한 건 이걸

"q가 16개니까 CU가 16개"

라고 바로 생각하면 안 된다는 거야.

각 CU가 자기 입력 lane을 하나씩 받아서 **동일한 BConv 계산을 병렬 수행하는 구조**라고 먼저 이해하는 게 안전하다.

coeff[0] —→ CU0

coeff[1] —→ CU1

coeff[2] —→ CU2

...

coeff[15] —→ CU15

즉 SIMD lane처럼 생각하면 된다.

---

4. 이번에는 이 순서로 읽어

내가 네 입장이면 정확히 이렇게 읽는다.

단계	파일	같이 볼 TB	이해 목표
0	Bconv_engine.v	X	전체 지도만 보기
1	shoup_mult.v	tb_mult.v 계열	modular multiplication
2	CU.v	tb_CU.v	CU 하나의 STEP1/STEP2
3	Bconv_engine.v	tb_Bconv.v	CU ×16 + FSM
4	Python generator	generate_cu_test_vector.py 등	golden vector가 어떻게 만들어지는지
5	Bconv_test_vectors/	tb_Bconv.v	실제 regression
6	나머지 modular RTL	각각 TB	공용 arithmetic unit
7	automorphism	tb_automorphism.v	BConv와 별도 기능

처음에는 1→2→3만 해.

특히 modular_engine.sv, modular_engine_step1.sv, modular_engine_step2.sv 같은 것까지 한꺼번에 읽기 시작하면 다시 구조가 뒤섞인다.

---

5. 첫 번째 진짜 공부: shoup_mult.v

NTTU에서 butterfly가 첫 번째 핵심이었다면, BConv에서는 이게 첫 번째 핵심이다.

CU 안에 실제로:

```
shoup_mult #(
    .WORD_WIDTH(DATA_WIDTH),
    .K_WIDTH(DATA_WIDTH)
) shoup (
    ...
    .a              (mux_out),
```

```
        .ratio          (param),
        .z_pre          (ratio_precomp),
        .q              (q_in),

        .valid_out    (mult_done),
        .result       (modmult_out)
);
```

가 있다.

얘가 하는 건 간단히 말해서

₩[₩boxed{ ₩text{modmult₩_out} = ₩text{mux₩_out}₩times₩text{param} ₩bmod q } ₩]

야.

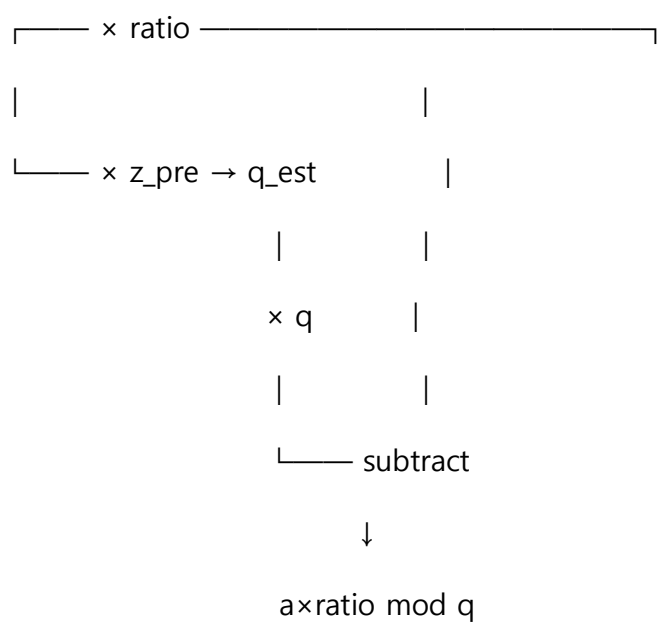
그런데 그냥 % q를 하는 게 아니라 **Shoup multiplication**을 쓴다.

미리

₩[z_{₩rm pre} = ₩left₩lfloor ₩frac{z2^K}{q} ₩right₩rfloor ₩]

를 계산해놓고,

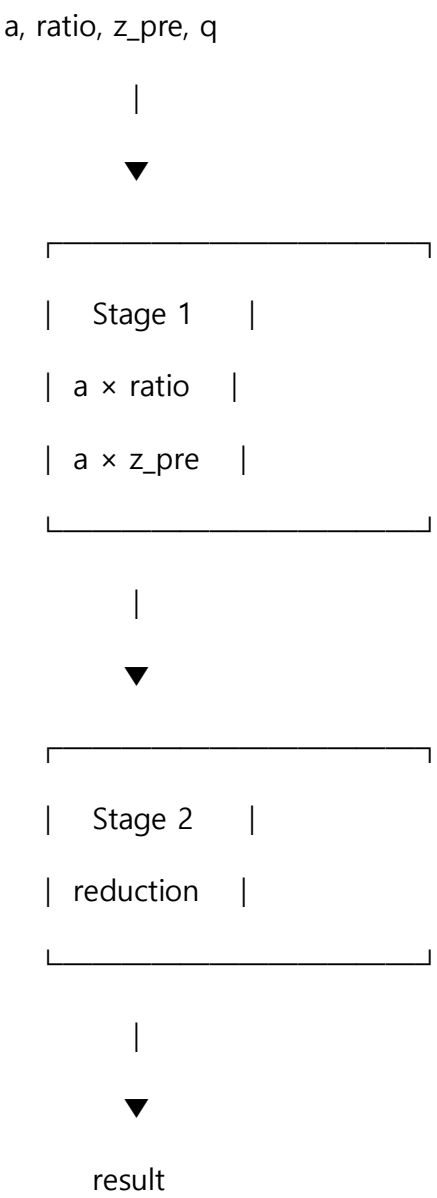
a



를 만든다.

그리고 이 RTL은 **2-stage pipeline**이다.

Cycle n



그래서

```
reg [1:0] vld;
...
vld <= {vld[0], valid_in};
assign valid_out = vld[1];
```

가 있는 거야.

여기서는 이것만 이해하면 돼

₩[a₩times param ₩pmod q ₩]

를 **2-cycle latency의 Shoup multiplier**로 계산한다.

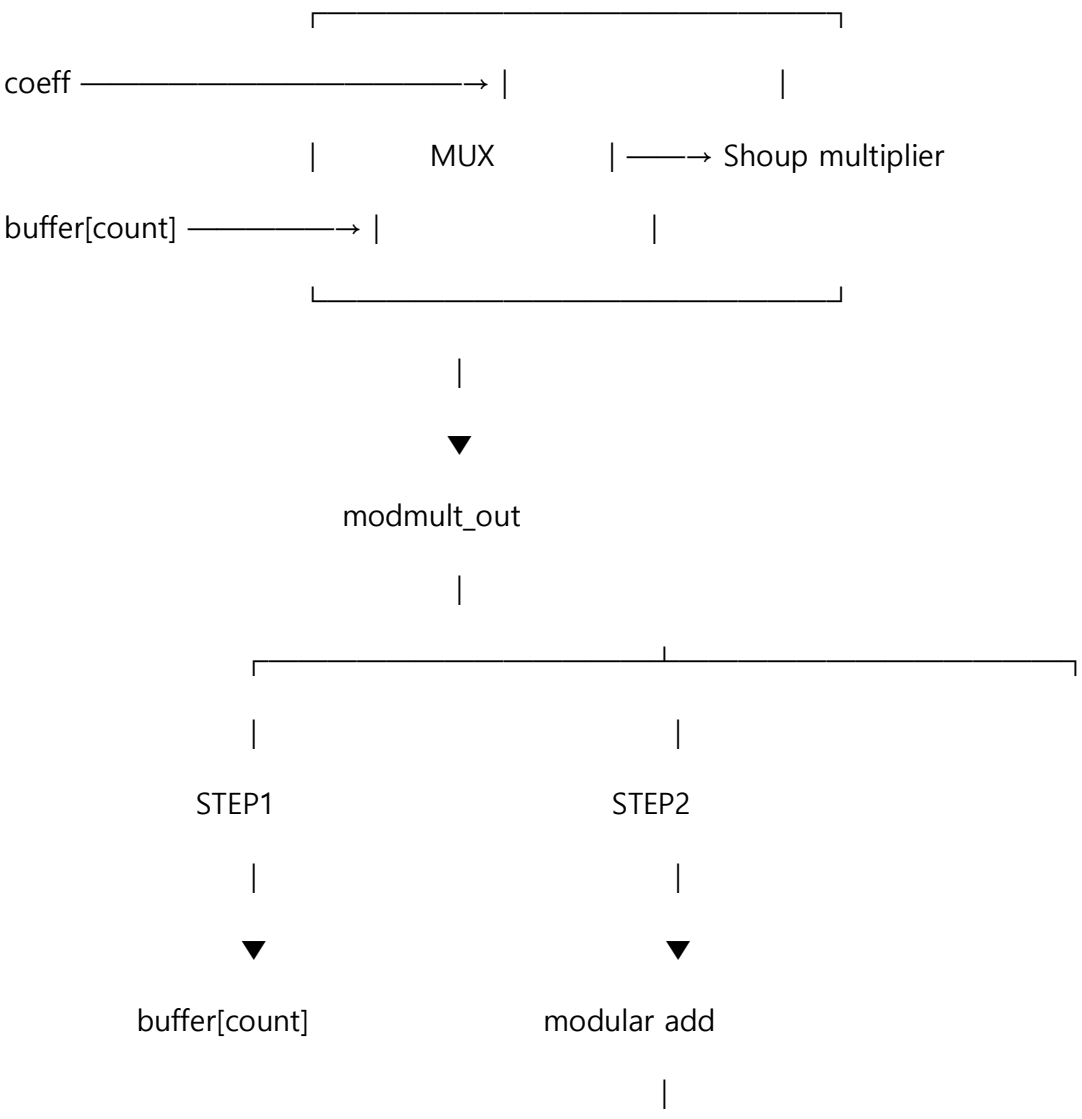
Shoup의 수학적 증명까지 처음부터 파지 마.

---

6. 그다음 CU.v가 진짜 핵심이다

이 파일은 집중해서 읽어야 한다.

CU 하나만 때면 구조가:



▼

cu_accum

|

▼

CU_out

이 구조를 머릿속에 넣고 코드를 보면 된다.

---

7. CU는 STEP1과 STEP2가 완전히 다른 일을 한다

이게 제일 중요하다.

STEP1

MUX를 보면:

```
if (en_step_1) begin
    mux_out = coeff;
end
```

그러니까

coeff

↓

Shoup multiplier

↓

buffer[]

이다.

실제로:

```
if (mult_done && en_step_1_dd) begin
    buffer[buffer_count_1] <= modmult_out;
end
```

이다.

수식으로 보면 대략

₩[b_i = coeff_i₩cdot param_i ₩pmod q ₩]

를 계산해서 저장하는 단계라고 보면 된다.

즉:

STEP1

coeff[0] × param → buffer[0]

coeff[1] × param → buffer[1]

coeff[2] × param → buffer[2]

...

coeff[L] × param → buffer[L]

이다.

여기서 level_in이 몇 개를 처리할지 결정한다.

---

8. STEP2에서는 buffer를 다시 읽는다

MUX가 바뀐다.

```
else if (en_step_2) begin
    mux_out = buffer[buffer_count_2];
end
```

즉 이번에는:

buffer[0]

buffer[1]

buffer[2]

...

↓

Shoup multiplier

↓

modmult_out

↓

누적

이다.

누적하는 부분이:

temp_sum =

modmult_out + cu_accum;

이고,

₩(q₩)보다 커지면:

(temp_sum >= q_in_dd) ?

(temp_sum - q_in_dd) :

temp_sum;

으로 modular reduction 한다.

따라서 논리적으로는:

₩[acc ₩leftarrow acc+ (buffer_i₩times param) ₩pmod q ₩]

가 반복되는 거야.

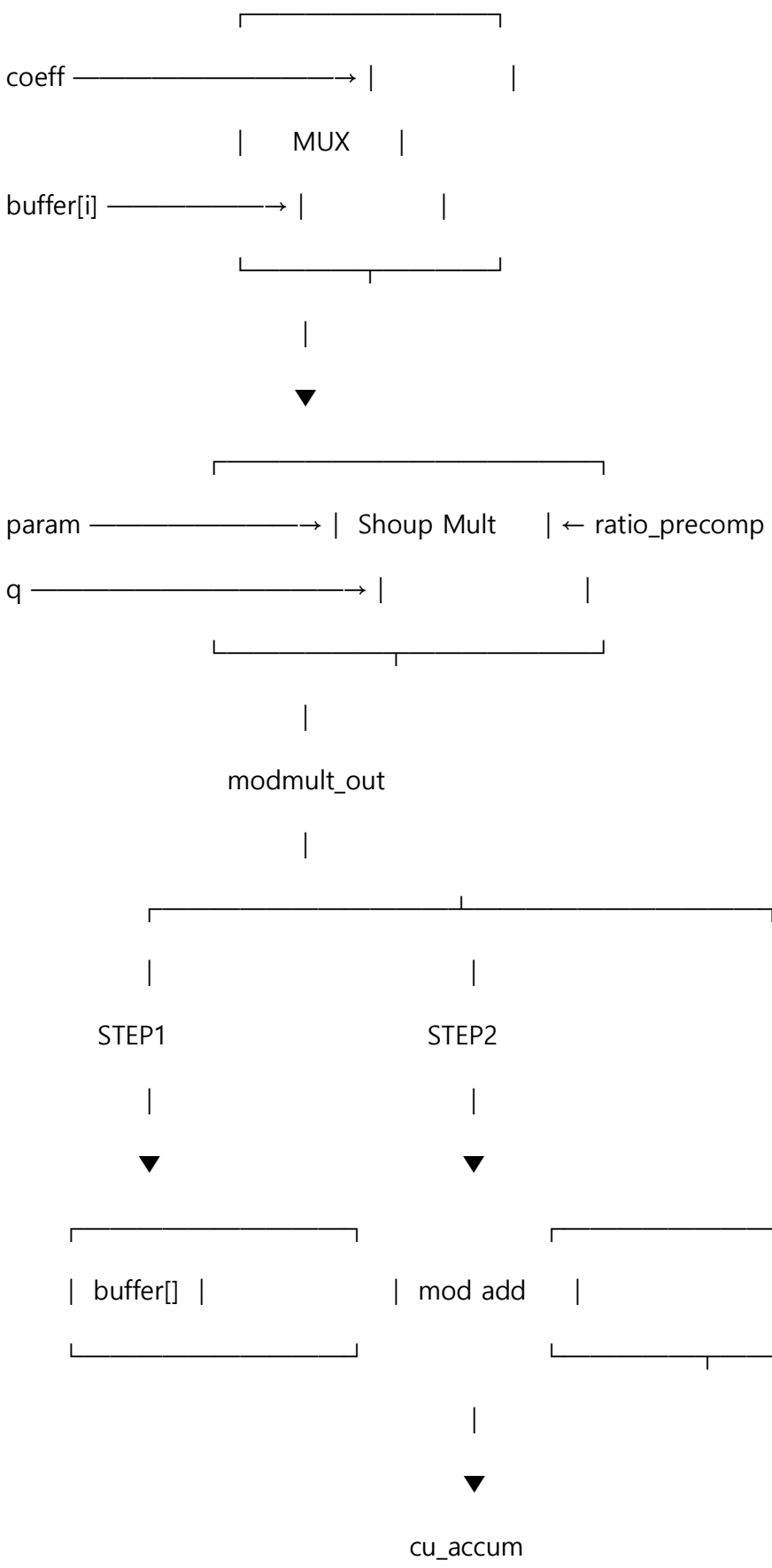
즉 **BConv의 CRT weighted sum**을 만드는 부분이 여기다.

---

9. STEP1 → STEP2를 한 장으로 보면

이게 이 RTL의 핵심 그림이야.

BConv CU


coeff
MUX
buffer[i]
param
Shoup Mult
ratio_precomp
q
modmult_out
STEP1
STEP2
buffer[]
mod add
cu_accum

|

▼

CU_out

이 그림을 이해하면 CU.v의 70~80%는 이해한 거다.

---

10. 그러면 Bconv_engine.v는 뭐 하냐?

오히려 CU보다 훨씬 쉽다.

크게 두 개밖에 없다.

① FSM

IDLE

↓ start_Bconv

STEP1

↓

STEP2

↓

IDLE

상태 정의가:

IDLE = 00

STEP1 = 01

STEP2 = 10

이고

assign en_step_1 = (current_state == STEP1);

assign en_step_2 = (current_state == STEP2);

이므로 CU 16개 전체에:

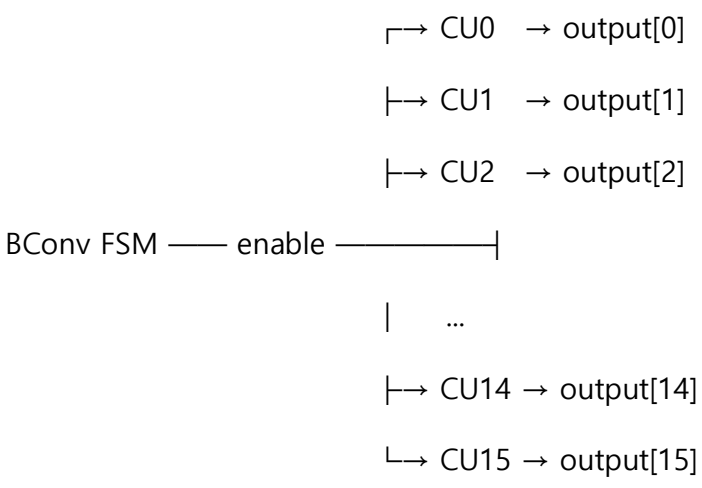
지금 buffer 만드는 중이다

또는

지금 accumulation 하는 중이다

라고 알려주는 controller다.

② CU 16개 병렬 연결



그래서 **Bconv_engine.v를 처음부터 깊게 읽을 필요가 없는 이유**가 이것이다.

실제 arithmetic은 거의 다 CU 안에 있다.

---

11. counter 세 개는 이렇게 구분해

Top에:

counter_1

counter_2

counter_3

세 개가 있어서 처음 보면 짜증나는데 역할을 크게 이렇게 잡으면 된다.

counter_1

STEP1 진행 횟수.

STEP1:

counter_1 = 0,1,2,...level_in

즉 입력 basis 쪽 residue들을 처리하는 진행도.

---

counter_2

STEP2 안에서 입력 level들을 순회.

0 → 1 → 2 → ... → level_in

                    |

                    └──── 다시 0

---

counter_3

counter_2가 한 바퀴 돌 때 증가한다.

즉 대략:

output basis #0

    q0 q1 q2 ... qL

output basis #1

    q0 q1 q2 ... qL

output basis #2

    q0 q1 q2 ... qL

...

같은 **2중 loop를 FSM counter로 구현한 구조**로 보면 된다.

개념적으로:

STEP 1for i in input_basis: buffer[i] = ...# STEP 2for j in output_basis: acc = 0 for i in input_basis: acc += buffer[i] * ... output[j] = acc

이걸 하드웨어 counter로 바꾼 게

counter_2 ≈ inner loop

counter_3 ≈ outer loop

라고 보면 이해가 빠르다.

---

12. 다만 여기서 코드 하나는 반드시 직접 확인해야 한다

CU.v 마지막이 좀 특이하다.

```
always @(buffer_count_2) begin
    if ((en_step_2_ddd) && (buffer_count_2 == 4'd2)) begin
        CU_out = cu_accum;
        valid_Bconv_out = 1;
```

즉 현재 코드상 **buffer_count_2 == 2일 때 output valid를 낸다.**

이게 일반적인

buffer_count_2 == level

형태가 아니다.

그래서 이건 내가 지금 일반 BConv 수학만 보고 의미를 만들어내면 안 되고, **tb_CU.v / generate_cu_test_vector.py를 같이 봐야 정확한 의도를 알 수 있는 부분**이야.

특히 네가 사수에게 코드 리뷰하면서 물어볼 만한 포인트다.

CU_out valid 조건이 buffer_count_2 == 2로 고정되어 있는데, 이 2가 pipeline latency alignment 때문에 들어간 건가요?

좋은 질문이다.

---

13. tb_CU.v부터 돌리는 게 제일 좋다

이번 폴더에서 첫 번째 제대로 된 실습은 **Bconv top이 아니라 CU 하나**다.

스크린샷에:

tb_CU.v

generate_cu_test_vector.py

generate_cu_vectors_full.py

cu_test_vectors/

cu_test_vectors_full/

가 다 있잖아.

딱 봐도 제작자가 CU를 **독립 검증 가능하게 만들어놓은 것**이다.

Wave에는 처음에는 이것만 올려.

clk

rst_n

en_step_1

en_step_2

level

coeff

param

q_in

ratio_precomp

DUT.mux_out

DUT.modmult_out

DUT.mult_done

DUT.buffer_count_1

DUT.buffer_count_2

DUT.cu_accum

valid_Bconv_out

CU_out

그리고 처음에는 buffer[0], buffer[1], buffer[2] 정도만 추가해.

그러면 waveform에서:

STEP1

———————————————————

coeff ——————————————————┐

↓

Shoup

↓

buffer[0] ← result

buffer[1] ← result

buffer[2] ← result

...

STEP2

———————————————————

buffer[0] ——┐

buffer[1] ——┼→ Shoup → accumulation

buffer[2] ——┘

cu_accum

↓

CU_out

이 흐름이 보여야 한다.

---

14. 그다음 tb_Bconv.v

CU 하나가 이해된 다음에야 Bconv top을 돌린다.

Wave는 오히려 더 단순하게:

start_Bconv

current_state

counter_1

counter_2

counter_3

en_step_1

en_step_2

coeff[0]

param[0]

CU_inst_loop[0].CU_inst.modmult_out

CU_inst_loop[0].CU_inst.cu_accum

valid_Bconv_out[0]

CU_out[0]

처음부터 CU 16개 waveform에 다 올리지 마.

CU0 하나만 보면 된다.

16개는 같은 구조를 병렬 복제한 것이기 때문이다.

CU0가 이해되고 나서:

CU_out[0]

CU_out[1]

...

CU_out[15]

만 마지막에 한 번 비교해.

---

15. Python 파일은 언제 읽냐?

이번에는 Python도 꽤 중요하다.

추천 순서는:

CU.v

↓

tb_CU.v

↓

generate_cu_test_vector.py

다.

왜냐하면 RTL만 보면

coeff

param

q_in

ratio_precomp

가 **수학적으로 정확히 무엇을 의미하는지 이름만으로 100% 확정하기 어렵다.**

Python generator를 보면 대개:

ratio_precomp = ...param = ...expected = ...

형태로 golden result를 만들 테니까,

그때

₩[param = ? ₩]₩[ratio₩_precomp = ₩left₩lfloor₩frac{param₩cdot2^W}{q}₩right₩rfloor ₩]

같은 관계가 확실하게 드러난다.

즉 **RTL = 어떻게 계산하는가**,
Python generator = 무엇을 계산하는가

를 알려주는 파일이라고 생각하면 돼.

---

16. barrett_mult.v는 왜 있는데 BConv는 Shoup을 쓰지?

이것도 좋은 포인트다.

현재 CU에서는 명백히:

shoup_mult

를 instantiate하고 있다.

Shoup은 특히 **곱해지는 한쪽 값이 미리 알려진 constant**일 때 유리하다.

BConv에는 CRT에서 사용하는 각종 precomputed constant가 많다.

그러니까:

₩[a₩times C₩pmod q ₩]

에서 ₩(C₩)에 대해

₩[C_{₩rm pre} = ₩left₩lfloor ₩frac{C2^W}{q} ₩right₩rfloor ₩]

까지 미리 만들어놓고 빠르게 modular multiplication 하는 구조와 잘 맞는다.

반면 barrett_mult.v는 좀 더 일반적인:

₩[A₩times B₩bmod q ₩]

용으로 생각하면 된다.

NTTU에서도 Shoup와 Barrett이 서로 다른 역할을 했고, 특히 OF-Twist에서는 Shoup 쪽이 미리 계산된 상수를 이용한 TF sequence와 관련되고 Barrett 쪽이 실제 데이터 곱에 쓰였다. rtl

---

17. modular_unit.v는 당장은 빼놔

이 파일을 보면:

mode 000 idle

mode 001 add

mode 010 sub

mode 011 mult

mode 100 rot

mode 101 addrot

이다.

즉 얘는 **BConv CU 전용 모듈이 아니라 공용 modular arithmetic execution unit**에 더 가깝다.

따라서 지금 BConv 공부하면서:

modular_unit

↓

modadd

↓

modsub

↓

barrett

↓

automorphism

까지 빠지면 또 산으로 간다.

BConv 이해가 끝난 다음 별도의 **Modular Unit 공부 세션**으로 떼는 게 좋다.

---

18. automorphism.v도 BConv 핵심이 아니다

얘는 현재:

product = i_in * k_in;

result_reg <= product[15:0];

이라서

₩[result=(i₩times k)₩bmod2^{16} ₩]

을 계산한다.

즉 coefficient/index permutation에 필요한 automorphism index 계산 쪽으로 보인다.

BConv arithmetic 자체와는 다른 축이다.

그러니까 지금은 무시해도 된다.

---

19. SRAM 1024×32도 당장 안 읽어도 된다

sram_1024w_32b.v는 ARM SRAM macro simulation model이라 파일도 엄청 길다.

절대로 이거 처음부터 읽지 마.

하드웨어 설계 관점에서는 그냥:

1024 words

×

32 bits

single-port SRAM

이라는 black box로 취급하면 된다.

나중에 어떤 상위 모듈에서:

무엇을 SRAM에 저장하는지

address가 어떻게 생성되는지

read/write가 언제 일어나는지

만 보면 된다.

SRAM 내부 Verilog 모델 자체를 공부할 이유는 거의 없다.

---

20. 결국 폴더 전체를 중요도별로 나누면

A급 — 반드시 직접 이해

Bconv_engine.v

CU.v

shoup_mult.v

tb_Bconv.v

tb_CU.v

generate_cu_test_vector.py

generate_cu_vectors_full.py

여기가 **BConv 본체**다.

B급 — BConv 이해 후

modular_engine.sv

modular_engine_step1.sv

modular_engine_step2.sv

modular_unit.v

modadd.v

modsub.v

barrett_mult.v

공용 modular datapath 쪽.

C급 — 별도 기능

automorphism.v

tb_automorphism.v

automorphism_test.py

Automorphism.

D급 — black box 취급

sram_1024w_32b.v

SRAM macro model.

---

21. NTTU와 BConv를 나란히 놓으면 훨씬 감이 온다

NTTU는 구조적으로:

input

↓

butterfly

↓

NTT16

↓

transpose

↓

twist

↓

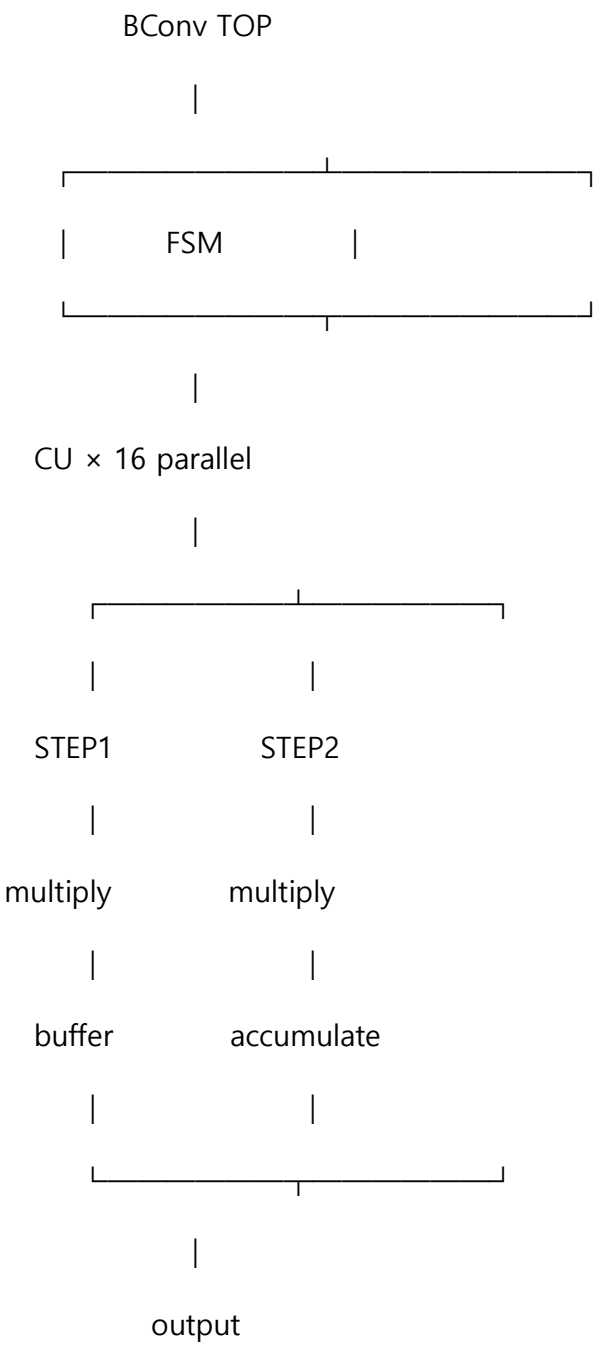
NTT16

↓

...

라서 **데이터 permutation + butterfly pipeline**이 어려웠다. 실제 RTL도 16-point NTT 하나에 radix-2 butterfly가 32개 들어가는 구조였다. rtl

BConv는 오히려:



이다.

그래서 **BConv 쪽이 NTTU보다 RTL 구조 자체는 훨씬 이해하기 쉬울 가능성이 높아.**

---

22. 내가 네 입장이면 실제로 이렇게 한다

첫날에 욕심내지 말고 딱:

① Bconv_engine.v 10분

→ CU 16개 있다는 것만 확인

② shoup_mult.v

→ input × param mod q

→ 2-cycle이라는 것 확인

③ CU.v

→ STEP1 : multiply → buffer

→ STEP2 : buffer → multiply → accumulate

④ tb_CU.v 실행

→ waveform으로 위 흐름 확인

여기까지만 한다.

그 다음:

⑤ generate_cu_test_vector.py

→ coeff/param/q/ratio_precomp의

정확한 수학적 의미 확인

⑥ tb_Bconv.v

→ FSM

→ counter 1/2/3

→ CU0 하나 추적

⑦ 16 CU output 비교

까지 가면 **BConv RTL의 핵심은 거의 끝난다.**

그 뒤에 modular_engine → modular_unit → add/sub/Barrett를 **두 번째 블록**으로 따로 보면 된다.

그리고 특히 이번 코드에서 네가 먼저 잡아야 하는 핵심 질문은 딱 세 개야.

① STEP1에서 정확히 어떤 CRT quantity를 만들어 buffer에 저장하는가?
② STEP2에서 그 buffer를 어떤 ₩(p_j₩)용 constant와 곱해서 어떻게 누적하는가?
③ 왜 valid_Bconv_out 조건이 buffer_count_2 == 2인가?

이 세 개를 CU.v + Python generator + tb_CU.v로 연결해서 이해하면, 그때부터는 단순히 Verilog를 읽는 게 아니라 **BConv 수식이 RTL의 어느 레지스터/곱셈기/counter로 구현됐는지** 보이기 시작할 거야.